Experiment 7

Batch Prediction System

Name: Kasyap Yanamandra Roll No: 231FA04E47 Date: 22 October 2026

1. Aim

To implement a batch inference system that applies a trained machine learning model to multiple input records in a single execution and stores the prediction results for further analysis.

2. Model and Dataset

Dataset: Breast Cancer Wisconsin (Diagnostic), loaded from scikit-learn (569 records, 30 numeric features). Target: 0 = malignant, 1 = benign.

Model: scikit-learn Pipeline = SimpleImputer (median) → StandardScaler → RandomForestClassifier (200 trees, random_state = 42). Training: 80/20 stratified split. Test accuracy = 0.9561, F1 = 0.9655.

Saved as models/model.joblib together with the model version (v1.0), feature names and class labels. The model is not retrained during batch inference.

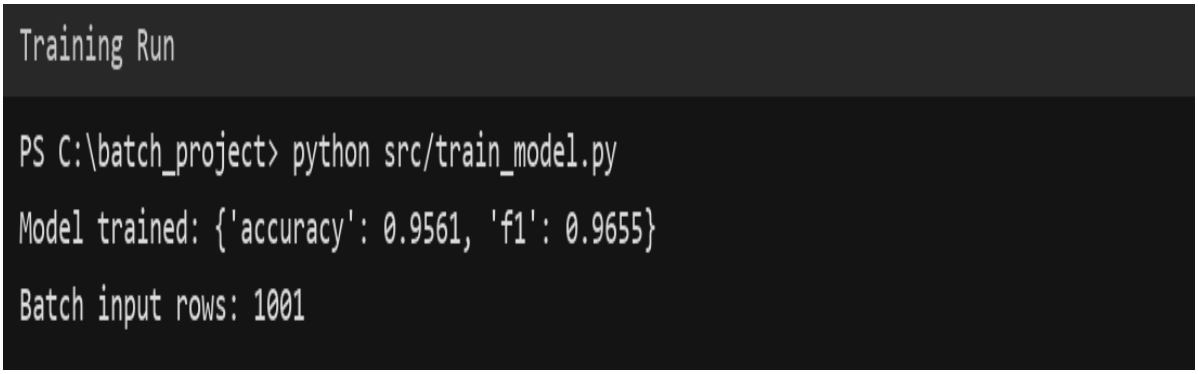


Figure 1: Model training output

3. Workflow and Architecture

Batch CSV → Load saved model → Validate records → Apply preprocessing pipeline → Predict all valid rows in one call → Save predictions.csv → Save performance.json and log.

Project structure:

project/
  data/batch_input.csv
  models/model.joblib, model_info.json
  outputs/predictions.csv, failed_records.csv, performance.json, batch_run.log
  src/train_model.py, batch_predict.py

Run command: python src/batch_predict.py --input data/batch_input.csv

4. Input Batch Dataset

The batch file contains 1001 rows: record_id plus the 30 feature columns. Four problems were deliberately inserted to test validation:

Row / Record	Problem	Expected handling
REC0006	Missing value in 'mean radius'	Accepted; filled by pipeline's median imputer
REC0011	Text 'abc' in 'mean area'	Rejected (invalid data type)
REC0021	Negative 'mean texture'	Rejected (invalid value)
REC0031 (last row)	Exact duplicate	Rejected (duplicate)

5. Batch Prediction Implementation

Reads the whole CSV with pandas and checks the schema (all 30 feature columns plus record_id must exist). Validation checks duplicates, non-numeric values and negative values. Failed rows are written to failed_records.csv with the reason; none are dropped silently.

The saved pipeline applies the same imputation and scaling used in training; all valid rows are predicted in a single predict() call. predict_proba() gives a confidence score (highest class probability) for each record.

Prediction timestamp and model version are stored with every row. Errors are logged in batch_run.log, and any fatal error exits with code 1.

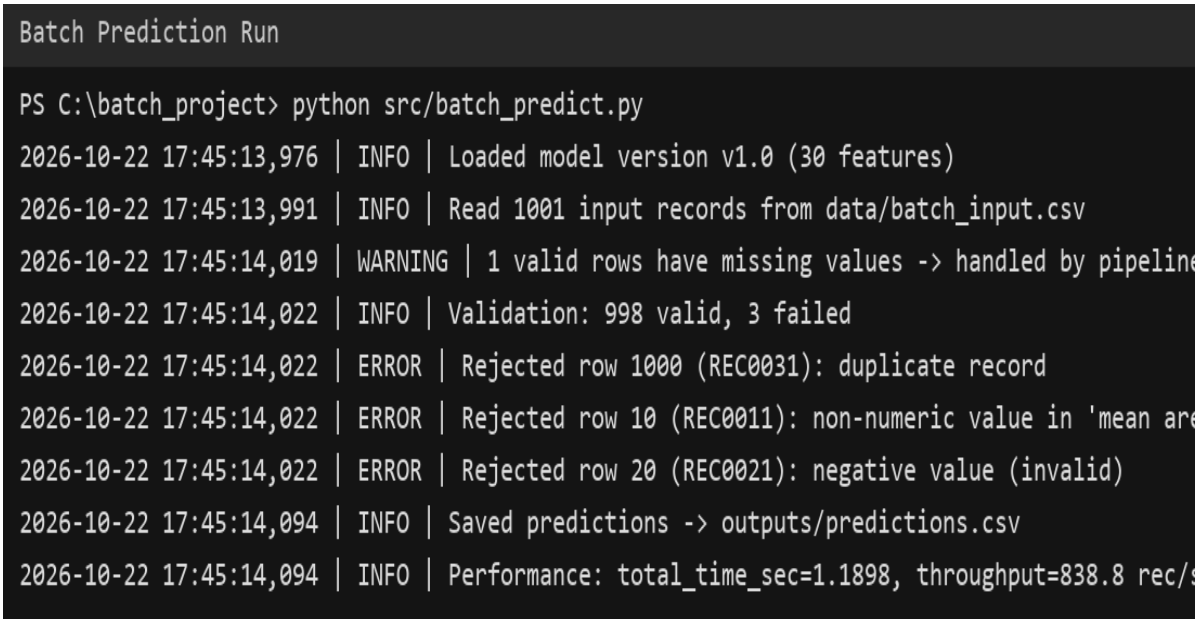
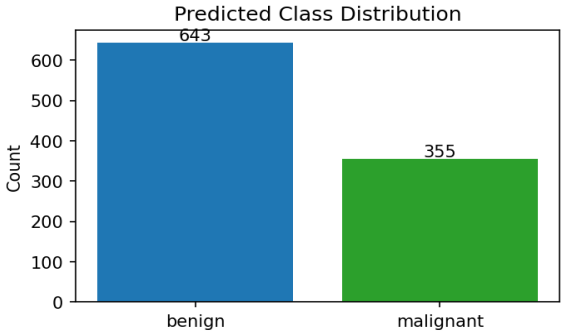


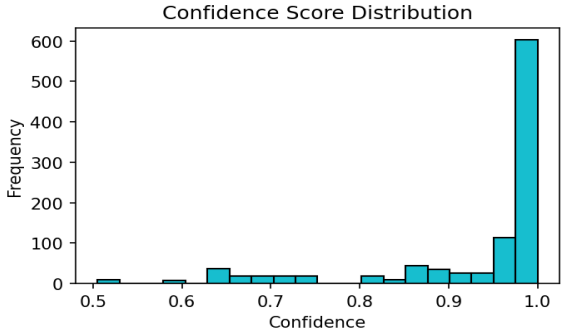
Figure 2: Batch prediction execution log (terminal screenshot)

6. Prediction Output

Output file: outputs/predictions.csv (998 rows). Columns: record_id, predicted_class, predicted_label, confidence, prediction_timestamp, model_version.

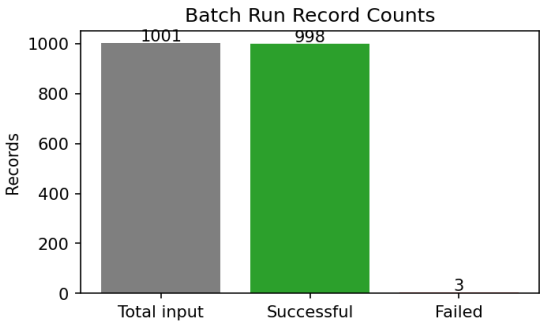
Prediction summary: 643 benign and 355 malignant. Records rejected: 3 (see failed_records.csv). Check: output rows (998) = input rows (1001) - failed rows (3). Verified as True.

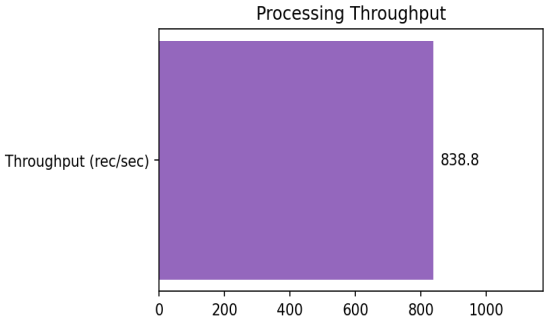




7. Performance Measurement

Metric	Value
Total input records	1001
Successful predictions	998
Failed records	3
Total time (load + validate + predict + save)	1.1898 s
Prediction-only time	0.0666 s
Average time per record	1.1886 ms
Throughput	838.8 records/sec
Model version	v1.0
Run timestamp	2026-10-22 17:45:14





Factors affecting batch performance: dataset size (more records = longer total time, but average time per record usually falls because fixed costs such as model loading are shared); preprocessing complexity (imputation, scaling add time per record); model complexity (Random Forest with more trees is slower than a linear model); I/O (reading/writing CSV; model loading and file I/O take most of the total time, since pure prediction took only 0.0666 s).

8. Batch vs Real-Time Prediction

Aspect	Batch prediction (Exp 7)	Real-time REST API (Exp 5)
Input pattern	Large file with many records at once	One record (or a few) per request
Execution frequency	Scheduled or on demand (daily, hourly)	Continuous, on every user request
Response requirement	Results needed later, no waiting user	Immediate response expected
Number of records	Hundreds to millions per run	Typically one per call
Latency	High per run, very low per record	Low per request (milliseconds)
Throughput	High (vectorised processing)	Limited by request handling overhead
Storage	Results written to file or database	Response returned to caller; logging optional

Batch inference is appropriate when no instant answer is needed (monthly patient risk screening, nightly scoring of records, report generation). Real-time inference is appropriate when a decision is needed at once (doctor entering values and expecting a result on screen). Batch inference can handle many records without one API request per record because the model and pipeline are loaded once and the whole table is transformed and predicted in one vectorised call.

9. Observations, Problems and Corrective Actions

• Observation: all 3 invalid records were caught and logged, and the output row count matched the successfully processed input count.

• Observation: the record with a missing value was still predicted, because the saved pipeline includes the imputer.

• Problem: when the bad batch file was first generated, the text value could not be placed in a numeric column. Corrective action: the column was converted to object type before inserting the invalid value.

• Problem: the first draft of the validation code crashed when no records failed. Corrective action: the failed-records table is now built so that it works with zero failures.

• Note: the batch input reuses test rows from the dataset, so it is for demonstrating the system rather than measuring true accuracy.

10. Conclusion

A batch prediction system was built that loads a saved model pipeline, validates 1001 input records, predicts 998 valid records in one execution, stores results with timestamps and model version, and reports performance (about 839 records/second). The comparison with real-time prediction shows that batch inference suits large, non-urgent workloads.